T1 观星（star）

双指针：固定左端点，复用区间统计

把 $a_i > q$ 的位置称为坏位置。固定左端点 L，向右扩展右端点，用 bad 维护区间内的坏位置数，找到坏位置数不超过 m 的最远右端点 R。若再加入一个位置就会超过 m，更远的右端点也一定不合法，便停止扩展。

当左端点从 L 移到 $L+1$ 时，删去最左边的位置不会增加坏位置数，所以 $[L,R]$ 满足限制，$[L+1,R]$ 也满足限制。**右端点不必重新枚举**：从 bad 中扣除被移出位置的贡献，再从当前 R 继续向右扩展即可；每加入一个坏位置，就把 bad 加一。

固定 L 后，右端点在 L 到 R 之间都满足坏位置数限制。要使长度至少为 k，还需右端点不小于 $L+k-1$，因此这一轮有

$$\max(0, R-L-k+2)$$

个合法区间。每个区间只在枚举到它的左端点时计入，累加各轮贡献就是答案。

初始取 $L=1, R=0, \text{bad}=0$，表示空窗口。若 $m=0$ 且 $a_L > q$，连第 L 个位置都不能加入，此时 $R=L-1$，贡献为 0。这个位置不在窗口内，无须从 bad 中扣除；将 R 置为 L，下一轮从空窗口 $[L+1, L]$ 继续。

两个指针各至多右移 n 次，总时间为 $O(n)$；代码保存原数组，空间为 $O(n)$。答案最多为 $n(n+1)/2$，需要用 long long 保存。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10;
int n,k,m,q;
int a[N];
int main(){
    freopen("star.in","r",stdin);
    freopen("star.out","w",stdout);
    cin >> n >> k >> m >> q;
    for(int i = 1; i <= n; i++) cin >> a[i];
    long long ans = 0;
    int r = 0,bad = 0;   // bad: 窗口 [l,r] 中的坏位置数
    for(int l = 1; l <= n; l++){
        while(r < n && bad + (a[r + 1] > q) <= m){
            r++;
            if(a[r] > q) bad++;
        }
        if(r - l + 1 >= k) ans += r - l - k + 2;
        if(l <= r){
            if(a[l] > q) bad--;
        }else{
            r = l;   // 当前窗口为空，下一轮从位置 l+1 开始
        }
    }
    cout << ans << endl;
    return 0;
}
```

部分分

每点 5 分，表中分值按题面保证的点号计算。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 100$	$1 \sim 4$，20 分	枚举长度至少为 k 的区间，扫描统计坏位置数，满足限制则计入答案。时间 $O(n^3)$，空间 $O(n)$。
2	$n \le 5000$	$1 \sim 10$，50 分	固定左端点向右扩展，增量统计坏位置数；未超过 m 且长度至少为 k 时计入答案，超过 m 时停止。每换一个左端点都重新枚举右端点，时间 $O(n^2)$，空间 $O(n)$。已覆盖上一档。
3	$m = 0$	$11 \sim 14$，20 分	坏位置把序列分成连续好段。长度为 $len \ge k$ 的好段贡献 $(len-k+1)(len-k+2)/2$，不足 k 则贡献 0；扫描时结算各段，并处理最后一段。时间 $O(n)$，代码保存序列，空间 $O(n)$。

$m = 0$ 时用分段公式，其余情况用二次做法，可保证覆盖点 $1 \sim 14$，共 70 分。

T2 史莱姆（slime）

先喂后分，再用根号枚举

设一共喂食 p 次、分裂 q 次。分裂不会产生更大的个体，每次喂食只让一只增大 1，所以每只史莱姆的大小至多为 $p+1$；分裂 q 次后恰有 $q+1$ 只，总大小至多为 $(p+1)(q+1)$。

先把最初的一只喂食 p 次，再分裂 q 次，就能达到这个上界，花费仍是 $ap+bq$。因此，对任意可行方案，都可以用相同花费的先喂后分方案替代。

令最终每只大小为 $x=p+1$，只数为 $y=q+1$，需要求

$$\min_{x,y\ge 1,\ xy\ge n}\big[a(x-1)+b(y-1)\big].$$

固定 x 后，因为 $b>0$，取最小可行的 $y=\lceil n/x\rceil$ 最优；固定 y 时同理取 $x=\lceil n/y\rceil$。

接下来利用乘积关系缩小枚举范围。若 $x>\sqrt{n}$，那么 $n/x<\sqrt{n}$，它对应的最优只数满足

$$y=\left\lceil\frac{n}{x}\right\rceil\le\lceil\sqrt{n}\rceil.$$

若 $x\le\sqrt{n}$，则 x 本身就在这个范围内。因此，**最优方案中，x,y 至少有一个不超过 $\lceil\sqrt{n}\rceil$**：一侧取大时，另一侧只需取到根号范围；继续增大另一侧只会多花钱。

于是分别枚举根号范围内的 x 和 y，每次让另一侧取最小可行值，计算花费并取最小值。由于 a,b 可能不同，较小的一侧可能是 x，也可能是 y，两侧都要枚举。

实现时才令 $s=\lfloor\sqrt{n}\rfloor$，把两次枚举的上界统一写成 $s+1$。它一定覆盖 $\lceil\sqrt{n}\rceil$；n 为完全平方数时只是多检查一个候选，不会漏掉最优解。

复杂度与实现

单组时间 $O(\sqrt{n})$，额外空间 $O(1)$；多组总时间 $O(\sum_t\sqrt{n_t})$。当 $n=10^{12}$ 时，每组约检查 2×10^6 个候选。

向上取整用整数式 $(n+x-1)/x$。代码先用 sqrt 求近似值，再用整数乘法修正 s，使 $s^2\le n<(s+1)^2$。候选总代价小于 2×10^{18}，乘法和加法均用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
int T;
long long n,a,b;
int main(){
    freopen("slime.in","r",stdin);
    freopen("slime.out","w",stdout);
    cin >> T;
    while(T--){
        cin >> n >> a >> b;
        // s 为 n 的整数平方根：s * s <= n < (s + 1) * (s + 1)
        long long s = (long long)sqrt((double)n);
        while(s * s > n) s--;
        while((s + 1) * (s + 1) <= n) s++;
        long long ans = -1;
        // 最终有 y 只史莱姆、每只大小为 x，需要 x * y >= n，花费 a(x-1) + b(y-1)
        // 情况一：x <= s + 1，枚举 x，y 取最小的 ceil(n / x)
        for(long long x = 1; x <= s + 1; x++){
            long long y = (n + x - 1) / x;
            long long cost = a * (x - 1) + b * (y - 1);
            if(ans == -1 || cost < ans) ans = cost;
        }
        // 情况二：x > s + 1，此时 y <= s + 1，枚举 y，x 取最小的 ceil(n / y)
        for(long long y = 1; y <= s + 1; y++){
            long long x = (n + y - 1) / y;
            long long cost = a * (x - 1) + b * (y - 1);
            if(cost < ans) ans = cost;
        }
        cout << ans << endl;
    }
    return 0;
}
```

部分分

每点 5 分，下表列出各算法的保证覆盖。

序号	条件	点号与保证分	做法与复杂度
1	每组 $n \le 1000$	$1 \sim 3$，15 分	只喂食或复制最大个体，仍包含正文的先喂后分最优方案。设 $f[s][v]$ 为总大小记作 s、最大个体大小为 v 时的最小代价，达到目标的总大小统一记作 n。初始 $f[1][1]=0$，其余为无穷大；按 s 递增，从 $s<n$ 的状态用 $f[s][v]+a$ 更新 $(s+1,v+1)$，用 $f[s][v]+b$ 更新 $(\min(n,s+v),v)$，答案为 $\min_v f[n][v]$。单组时间、空间均为 $O(n^2)$。
2	每组 $n \le 10^6$	$1 \sim 9$，合计 45 分	枚举 $x=1,\dots,n$，取 $y=\lceil n/x \rceil$，计算花费并取最小值；$x>n$ 比 $(n,1)$ 更贵，无须枚举。单组时间 $O(n)$、空间 $O(1)$，已覆盖上一档。
3	每组 $a \ge b$	$10 \sim 13$，20 分	若 $x>y$，交换两者后花费的变化为 $(a-b)(y-x) \le 0$。正文已证明可选出至少一侧不超过 $s+1$ 的最优解，再把较小的一侧作为 x，故只需枚举 $x=1,\dots,s+1$，取 $y=\lceil n/x \rceil$。单组时间 $O(\sqrt{n})$、空间 $O(1)$。

按每组输入分派：$a \ge b$ 时用单侧根号枚举，否则用线性枚举，保证覆盖点 $1 \sim 13$，共 65 分。一个测试点内的所有数据组都必须完成，才能取得该点分数。

T3 旋律（melody）

满分思路：搜索下一位接谁

可以先搜索上行片段依次保留哪些数。任选一个位置作为片段的开头，然后不断决定：**下一个数接谁？**片段前后的数可以保留，不影响这一段的连续性，因此只需计算片段内部的删除次数。

假设当前选到了第 i 个数，片段内部已经删了 j 个数，长度为 len。如果下一位选第 p 个数，就要满足：

- $p > i$ 且 $a_p > a_i$，这样顺序正确，并且严格上升；
- 把两者中间的 $g = p - i - 1$ 个数全部删掉，才能让它们在新序列中相邻；
- $j + g \le k$，删除次数不能超限。

选定后，搜索就从 (i, j, len) 走到 $(p, j + g, len + 1)$；也可以在当前位置结束，得到长度为 len 的片段。任意合法片段都能按其保留位置从左到右这样选出，因此不会漏解。

如果两条搜索路径都走到了同一个 (i, j)，后面能接哪些位置、还能删几个数都完全相同。此时只需要保留当前长度较大的路径：较短路径能接上的后续，较长路径也能接上，最终长度不会更小。

于是得到状态：$f[i][j]$ **表示第 i 个数已被选作片段末尾，片段内部已经恰好删了 j 个数时，能得到的最大长度**。不必记录此前具体选了哪些位置。

沿用搜索中“接上第 p 个数”这一步，把当前最优长度加 1，更新到新的状态：

$$f[p][j+g] \leftarrow \max\big(f[p][j+g], f[i][j] + 1\big), \qquad p = i + g + 1.$$

只从可达状态出发，枚举 $0 \le g \le k - j$，并检查 $p \le n$ 和 $a_i < a_p$。

每个位置都可以单独作为开头，所以初始化 $f[i][0] = 1$；其余状态为 0，表示暂时不可达。每次都从较小的位置 i 更新到较大的位置 p，按 i 从小到大处理时，所有能到达 i 的路径都已处理完。片段可以随时结束，答案就是所有 $f[i][j]$（$0 \le j \le k$）的最大值。

例如，$a = (1, 5, 2, 3, 4, 2)$、$k = 1$ 时，保留位置 $1, 3, 4, 5$ 和保留位置 $3, 5$，都会到达 $(5, 1)$，长度分别为 4 和 2。合并到达这一状态的路径时，只保留长度 4，即 $f[5][1] = 4$。

复杂度

共有 $n(k+1)$ 个状态。对于已删 j 个数的状态，至多检查 $k - j + 1$ 个下一位置，总检查次数不超过

$$n \sum_{j=0}^{k} (k - j + 1) = n \frac{(k+1)(k+2)}{2}.$$

时间复杂度为 $O(n(k+1)^2)$，最大约 4.62×10^7 次检查；空间复杂度为 $O(n(k+1))$，代码中的数组共占约 20 MiB。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10;
const int K = 25;
int n,k;
int a[N];
// f[i][j]: 以 i 结尾、片段内部恰好删去 j 个数的最大长度
int f[N][K];
int main(){
    freopen("melody.in","r",stdin);
    freopen("melody.out","w",stdout);
    cin >> n >> k;
    for(int i = 1; i <= n; i++) cin >> a[i];
    for(int i = 1; i <= n; i++) f[i][0] = 1;
    int ans = 1;
    for(int i = 1; i <= n; i++){
        for(int j = 0; j <= k; j++){
            if(f[i][j] == 0) continue;
            ans = max(ans,f[i][j]);
            for(int g = 0; g <= k - j; g++){
                int p = i + g + 1;
                if(p > n) break;
                if(a[i] < a[p]){
                    f[p][j + g] = max(f[p][j + g],f[i][j] + 1);
                }
            }
        }
    }
    cout << ans << endl;
    return 0;
}
```

部分分

每点 5 分；下表列出各算法的保证覆盖。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 18$ 或 $k = 0$	$1 \sim 6$，30 分	依次搜索每个数删或留，删除次数达到 k 后只保留；到末尾扫描剩余序列的最长严格上升连续段。一般时间 $O(2^n n)$；$k = 0$ 时只有一条路径，时间 $O(n)$。空间 $O(n)$。
2	$k = 0$	$4 \sim 6$，15 分	扫描并维护当前上升段长：若 $a_{i-1} < a_i$ 就加一，否则从 1 重新开始，全程取最大值。时间 $O(n)$；随包程序保存原数组，空间 $O(n)$。
3	$k \le 1$	$4 \sim 10$，35 分	预处理以 i 结尾、开头的最长上升段长 l_i, r_i。先取不删时的最大值；$k = 1$ 时枚举 $2 \le i < n$，若 $a_{i-1} < a_{i+1}$，用 $l_{i-1} + r_{i+1}$ 更新答案。时间、空间均为 $O(n)$。

至多删一个数时，只有跨过删除位置的片段可能增长，两侧分别取最长上升段即可；不跨过删除位置的片段和删除首尾的情况都不会超过不删时的答案。

搜索已覆盖 $k = 0$。在 $n \le 18$ 时用搜索、其余在 $k \le 1$ 时用左右上升段，联合覆盖 $1 \sim 10$ 点，共 50 分；重叠点只计一次。

T4 合唱（chorus）

满分思路：每段改成中位数

把圆圈分成若干段，每段至少有 L 个数，再把每段改成同一个音高。相邻两段若音高相等，会合并成更长的声部，仍满足长度要求；任意合法结果也能按声部分段。因此，只需最小化这种分段的总代价，不必限制相邻段的音高不同。

一段内的音高排序后为 $b_1 \le \cdots \le b_t$。将首尾配对，对公共音高 c 有

$$|b_1 - c| + |b_t - c| \ge b_t - b_1,$$

当 $c \in [b_1, b_t]$ 时取等号。其余配对同理，中位数 $b_{\lceil t/2 \rceil}$ 落在所有配对区间内，能让每对同时取得最小值；奇数长度剩下的中间数也恰好贡献 0。所以每段改成中位数最优。

限制段长与段首

若一段的长度 $t \ge 2L$，把它切成长度为 L 和 $t - L$ 的两段。两段先沿用原来的音高，代价不变；分别改成各自的中位数后，代价只会减少或不变。从一个最优方案出发，不断切开过长的段，就能得到**每段长度都在 $[L, 2L-1]$ 内的最优方案**。

设 $M = 2L - 1$。在这样的最优方案中，若位置 1 是段首，直接从这里展开圆圈；否则，包含位置 1 的段至多长 M，从位置 1 顺时针走至多 $M - 1$ 步就到下一段的开头。因此，前 M 个位置中必有一个可用段首。只有一段时，可以把段首放在位置 1；若 $n < M$，尝试全部 n 个位置即可。

所以只需尝试前 $\min(n, 2L-1)$ 个段首。某次展开会保留一个最优分段，而展开后得到的每个合法分段放回圆圈也都合法，取这些答案的最小值即可。

实现时复制数组，令 $a_{i+n} = a_i$；枚举从位置 $s+1$ 展开，其中

$$0 \le s \le \min(n-1, 2L-2).$$

代码中的 $\mathsf{maxT} = 2L - 1$，maxS 是最大的 s。

前缀 DP

设 $\mathrm{cost}(i, t)$ 为复制数组中从位置 i 开始、长度为 t 的一段改成中位数的代价。固定 s 后，令 $f[j]$ 为**将 $a_{s+1}, \ldots, a_{s+j}$ 全部分成合法短段的最小代价**。

枚举最后一段的长度 t，它从位置 $s + j - t + 1$ 开始，前面还剩 $j - t$ 个数。各段代价独立相加，前缀只需保留最小代价，因此

$$f[j] = \min_{L \le t \le \min(2L-1, j)} \big(f[j-t] + \mathrm{cost}(s + j - t + 1, t) \big).$$

每种合法分段都能按最后一段归入某个 t，而合法前缀接上这一段也仍然合法。初始化 $f[0] = 0$，其余为无穷大，只从可行状态转移；按 j 递增计算，当前段首对应的答案就是 $f[n]$。

例如 $L = 2$，展开后为 $(1, 7, 2, 9, 3, 8)$。前 3 个数只能分成一段，代价为 $f[3] = 6$；前 4 个数分成两个长为 2 的段，代价为 $f[4] = 6 + 7 = 13$。计算 $f[6]$ 时，最后一段长 3 或 2，代价分别为 6 和 5，所以 $f[6] = \min(6 + 6, 13 + 5) = 12$。

预处理与复杂度

对每个需要的起点 i，依次把 $a_i, a_{i+1}, \ldots$ 插入有序小数组，直到长度为 $2L-1$。每次插入花费 $O(L)$；长度达到 L 后，取中位数并扫描求绝对值之和，存入 $\mathrm{cost}(i,t)$。一个起点共用 $O(L^2)$ 时间。

代码预处理起点 $1 \sim n + \mathsf{maxS}$，段末端超过 $2n$ 时停止，预处理用时 $O(nL^2)$。DP 至多执行 $2L-1$ 次，每次计算 n 个状态、枚举 L 种段长，因此总时间仍为 $O(nL^2)$。空间为 $O(nL)$，静态数组合计约 67 MiB。

在 $n = 2 \times 10^5, L = 10$ 时，DP 至多检查 3.8×10^7 个候选段，预处理只操作长度至多 19 的小数组。把全圈改成音高 1 就有合法解，总代价小于 2×10^{14}，段代价和 DP 值用 long long 保存。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 4e5 + 50;
const long long INF = 4e18;
int n,L;
long long a[N];
// cost[i][t]：从位置 i 开始、长度为 t 的一段全部改成同一个数的最小代价（t 在 L ~ 2L-1 之间）
long long cost[N][20];
long long b[25];
long long f[N];
int main(){
    freopen("chorus.in","r",stdin);
    freopen("chorus.out","w",stdout);
    cin >> n >> L;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        a[i + n] = a[i];   // 复制一份，处理环
    }
    int maxT = 2 * L - 1;
    int maxS = min(maxT - 1,n - 1);   // 环上第一段的起点只需试 1 ~ maxS+1
    for(int i = 1; i <= maxS + n; i++){
        int m = 0;
        for(int t = 1; t <= maxT && i + t - 1 <= 2 * n; t++){
            // 把 a[i+t-1] 插入有序数组 b[1..m]
            long long x = a[i + t - 1];
            int p = ++m;
            while(p > 1 && b[p - 1] > x){
                b[p] = b[p - 1];
                p--;
            }
            b[p] = x;
            if(t >= L){
                long long mid = b[(t + 1) / 2],s = 0;   // 改成中位数代价最小
                for(int j = 1; j <= t; j++) s += abs(b[j] - mid);
                cost[i][t] = s;
            }
        }
    }
    long long ans = INF;
    for(int s = 0; s <= maxS; s++){
        // 从位置 s+1 起把环剪开，f[j]：前 j 个数分好段的最小代价
        f[0] = 0;
        for(int j = 1; j <= n; j++){
            f[j] = INF;
            for(int t = L; t <= maxT && t <= j; t++){
                if(f[j - t] == INF) continue;
                long long v = f[j - t] + cost[s + j - t + 1][t];
                if(v < f[j]) f[j] = v;
            }
        }
        if(f[n] < ans) ans = f[n];
    }
    cout << ans << endl;
    return 0;
}
```

部分分

每点 5 分；下表列出各算法的保证覆盖。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 10$	$1 \sim 2$，10 分	枚举非空的段首集合，相邻段首之间为一段；检查每段长度至少为 L，排序求中位数代价并求和，再取所有方案的最小值。只有一个段首时整圈为一段。时间 $O(2^n n \log n)$，空间 $O(n)$。
2	$L = 2$	$3 \sim 6$，20 分	每段只取 2 或 3 个数，代价分别为两数之差的绝对值和三数极差；尝试前 $\min(n, 3)$ 个段首，每次做前缀 DP。时间 $O(n)$；随包程序空间 $O(n)$。
3	$n \le 1000$	$1 \sim 2, 7 \sim 11$，35 分	预处理长度 $L \sim 2L-1$ 的段代价，尝试全部 n 个段首，每次做前缀 DP，取最小值。时间 $O(nL^2 + n^2 L)$，空间 $O(nL)$。

全部段首的做法已覆盖 $n \le 10$。在 $n \le 1000$ 时用它、其余在 $L = 2$ 时用专用做法，联合覆盖 $1 \sim 11$ 点，共 55 分；重叠点只计一次。